

logic

February 27, 2025

1 Logic

This Jupyter notebook acts as supporting material for topics covered in **Chapter 6 Logical Agents**, **Chapter 7 First-Order Logic** and **Chapter 8 Inference in First-Order Logic** of the book *Artificial Intelligence: A Modern Approach*. We make use of the implementations in the `logic.py` module. See the [intro notebook](#) for instructions.

Let's first import everything from the `logic` module.

```
[100]: from utils import *
      from logic import *
      from notebook import psource
```

1.1 CONTENTS

- Logical sentences
 - Expr
 - PropKB
 - Knowledge-based agents
 - Inference in propositional knowledge base
 - * Truth table enumeration
 - * Proof by resolution
 - * Forward and backward chaining
 - * DPLL
 - * WalkSAT
 - * SATPlan
 - FolKB
 - Inference in first order knowledge base
 - * Unification
 - * Forward chaining algorithm
 - * Backward chaining algorithm

1.2 Logical Sentences

The `Expr` class is designed to represent any kind of mathematical expression. The simplest type of `Expr` is a symbol, which can be defined with the function `Symbol`:

```
[101]: Symbol('x')
```

```
[101]: x
```

Or we can define multiple symbols at the same time with the function `symbols`:

```
[102]: (x, y, P, Q, f) = symbols('x, y, P, Q, f')
```

We can combine `Exprs` with the regular Python infix and prefix operators. Here's how we would form the logical sentence "P and not Q":

```
[103]: P & ~Q
```

```
[103]: (P & ~Q)
```

This works because the `Expr` class overloads the `&` operator with this definition:

```
python def __and__(self, other): return Expr('&', self, other)
```

and does similar overloads for the other operators. An `Expr` has two fields: `op` for the operator, which is always a string, and `args` for the arguments, which is a tuple of 0 or more expressions. By "expression," I mean either an instance of `Expr`, or a number. Let's take a look at the fields for some `Expr` examples:

```
[104]: sentence = P & ~Q
```

```
sentence.op
```

```
[104]: '&'
```

```
[105]: sentence.args
```

```
[105]: (P, ~Q)
```

```
[106]: P.op
```

```
[106]: 'P'
```

```
[107]: P.args
```

```
[107]: ()
```

```
[108]: Pxy = P(x, y)
```

```
Pxy.op
```

```
[108]: 'P'
```

```
[109]: Pxy.args
```

```
[109]: (x, y)
```

It is important to note that the `Expr` class does not define the *logic* of Propositional Logic sentences; it just gives you a way to *represent* expressions. Think of an `Expr` as an [abstract syntax tree](#). Each

of the `args` in an `Expr` can be either a symbol, a number, or a nested `Expr`. We can nest these trees to any depth. Here is a deeply nested `Expr`:

```
[110]: 3 * f(x, y) + P(y) / 2 + 1
```

```
[110]: (((3 * f(x, y)) + (P(y) / 2)) + 1)
```

1.3 Operators for Constructing Logical Sentences

Here is a table of the operators that can be used to form sentences. Note that we have a problem: we want to use Python operators to make sentences, so that our programs (and our interactive sessions like the one here) will show simple code. But Python does not allow implication arrows as operators, so for now we have to use a more verbose notation that Python does allow: `|'==>|` instead of just `==>`. Alternately, you can always use the more verbose `Expr` constructor forms:

Operation	Book	Python Infix Input	Python Out-put	Python Expr In-put
Negation	$\neg P$	<code>~P</code>	<code>~P</code>	<code>Expr('~', P)</code>
And	$P \ \& \ Q$	<code>P & Q</code>	<code>P & Q</code>	<code>Expr('&', P, Q)</code>
Or	$P \ \ Q$	<code>P Q</code>	<code>P Q</code>	<code>Expr(' ', P, Q)</code>
Inequality (Xor)	$P \ \wedge \ Q$	<code>P ^ Q</code>	<code>P ^ Q</code>	<code>Expr('^', P, Q)</code>
Implication	$P \rightarrow Q$	<code>P '==>' Q</code>	<code>P ==> Q</code>	<code>Expr('==>', P, Q)</code>
Reverse Implication	$Q \leftarrow P$	<code>Q '<==' P</code>	<code>Q <== P</code>	<code>Expr('<==', Q, P)</code>
Equivalence	$P \ \leftrightarrow \ Q$	<code>P '<=>' Q</code>	<code>P <=> Q</code>	<code>Expr('<=>', P, Q)</code>

Here's an example of defining a sentence with an implication arrow:

```
[111]: ~(P & Q) |'==>'| (~P | ~Q)
```

```
[111]: (~(P & Q) ==> (~P | ~Q))
```

1.4 `expr`: a Shortcut for Constructing Sentences

If the `|'==>|` notation looks ugly to you, you can use the function `expr` instead:

```
[112]: expr('~(P & Q) ==> (~P | ~Q)')
```

```
[112]: (~(P & Q) ==> (~P | ~Q))
```

`expr` takes a string as input, and parses it into an `Expr`. The string can contain arrow operators: `==>`, `<==`, or `<=>`, which are handled as if they were regular Python infix operators. And `expr` automatically defines any symbols, so you don't need to pre-define them:

```
[113]: expr('sqrt(b ** 2 - 4 * a * c)')
```

```
[113]: sqrt(((b ** 2) - ((4 * a) * c)))
```

For now that's all you need to know about `expr`. If you are interested, we explain the messy details of how `expr` is implemented and how `|'==>|` is handled in the appendix.

1.5 Propositional Knowledge Bases: `PropKB`

The class `PropKB` can be used to represent a knowledge base of propositional logic sentences.

We see that the class `KB` has four methods, apart from `__init__`. A point to note here: the `ask` method simply calls the `ask_generator` method. Thus, this one has already been implemented, and what you'll have to actually implement when you create your own knowledge base class (though you'll probably never need to, considering the ones we've created for you) will be the `ask_generator` function and not the `ask` function itself.

The class `PropKB` now. * `__init__(self, sentence=None)`: The constructor `__init__` creates a single field `clauses` which will be a list of all the sentences of the knowledge base. Note that each one of these sentences will be a 'clause' i.e. a sentence which is made up of only literals and ors. * `tell(self, sentence)`: When you want to add a sentence to the KB, you use the `tell` method. This method takes a sentence, converts it to its CNF, extracts all the clauses, and adds all these clauses to the `clauses` field. So, you need not worry about telling only clauses to the knowledge base. You can `tell` the knowledge base a sentence in any form that you wish; converting it to CNF and adding the resulting clauses will be handled by the `tell` method. * `ask_generator(self, query)`: The `ask_generator` function is used by the `ask` function. It calls the `tt_entails` function, which in turn returns `True` if the knowledge base entails query and `False` otherwise. The `ask_generator` itself returns an empty dict `{}` if the knowledge base entails query and `None` otherwise. This might seem a little bit weird to you. After all, it makes more sense just to return a `True` or a `False` instead of the `{}` or `None`. But this is done to maintain consistency with the way things are in First-Order Logic, where an `ask_generator` function is supposed to return all the substitutions that make the query true. Hence the dict, to return all these substitutions. I will be mostly be using the `ask` function which returns a `{}` or a `False`, but if you don't like this, you can always use the `ask_if_true` function which returns a `True` or a `False`. * `retract(self, sentence)`: This function removes all the clauses of the sentence given, from the knowledge base. Like the `tell` function, you don't have to pass clauses to remove them from the knowledge base; any sentence will do fine. The function will take care of converting that sentence to clauses and then remove those.

1.6 Wumpus World KB

Let us create a PropKB for the wumpus world with the sentences mentioned in section 7.4.3.

```
[114]: wumpus_kb = PropKB()
```

We define the symbols we use in our clauses. $P_{x,y}$ is true if there is a pit in $[x, y]$. $B_{x,y}$ is true if the agent senses breeze in $[x, y]$.

```
[115]: P11, P12, P21, P22, P31, B11, B21 = expr('P11, P12, P21, P22, P31, B11, B21')
```

Now we tell sentences based on section 7.4.3. There is no pit in $[1,1]$.

```
[116]: wumpus_kb.tell(~P11)
```

A square is breezy if and only if there is a pit in a neighboring square. This has to be stated for each square but for now, we include just the relevant squares.

```
[117]: wumpus_kb.tell(B11 | '<=>' | ((P12 | P21)))
wumpus_kb.tell(B21 | '<=>' | ((P11 | P22 | P31)))
```

Now we include the breeze percepts for the first two squares leading up to the situation in Figure 7.3(b)

```
[118]: wumpus_kb.tell(~B11)
wumpus_kb.tell(B21)
```

We can check the clauses stored in a KB by accessing its `clauses` variable

```
[119]: wumpus_kb.clauses
```

```
[119]: [~P11,
      (~P12 | B11),
      (~P21 | B11),
      (P12 | P21 | ~B11),
      (~P11 | B21),
      (~P22 | B21),
      (~P31 | B21),
      (P11 | P22 | P31 | ~B21),
      ~B11,
      B21]
```

We see that the equivalence $B_{1,1} \iff (P_{1,2} \vee P_{2,1})$ was automatically converted to two implications which were in turn converted to CNF which is stored in the KB. $B_{1,1} \iff (P_{1,2} \vee P_{2,1})$ was split into $B_{1,1} \implies (P_{1,2} \vee P_{2,1})$ and $B_{1,1} \impliedby (P_{1,2} \vee P_{2,1})$. $B_{1,1} \implies (P_{1,2} \vee P_{2,1})$ was converted to $P_{1,2} \vee P_{2,1} \vee \neg B_{1,1}$. $B_{1,1} \impliedby (P_{1,2} \vee P_{2,1})$ was converted to $\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1}$ which becomes $(\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$ after applying De Morgan's laws and distributing the disjunction. $B_{2,1} \iff (P_{1,1} \vee P_{2,2} \vee P_{3,2})$ is converted in similar manner.

1.7 Knowledge based agents

A knowledge-based agent is a simple generic agent that maintains and handles a knowledge base. The knowledge base may initially contain some background knowledge. The purpose of a KB agent is to provide a level of abstraction over knowledge-base manipulation and is to be used as a base class for agents that work on a knowledge base. Given a percept, the KB agent adds the percept to its knowledge base, asks the knowledge base for the best action, and tells the knowledge base that it has in fact taken that action. Our implementation of KB-Agent is encapsulated in a class `KBAgentProgram` which inherits from the `KB` class. Let's have a look.

```
[120]: psource(KBAgentProgram)
```

<IPython.core.display.HTML object>

The helper functions `make_percept_sentence`, `make_action_query` and `make_action_sentence` are all aptly named and as expected, `make_percept_sentence` makes first-order logic sentences about percepts we want our agent to receive, `make_action_query` asks the underlying KB about the action that should be taken and `make_action_sentence` tells the underlying KB about the action it has just taken.

1.8 Inference in Propositional Knowledge Base

In this section we will look at two algorithms to check if a sentence is entailed by the KB. Our goal is to decide whether $KB \models \alpha$ for some sentence α . **### Truth Table Enumeration** It is a model-checking approach which, as the name suggests, enumerates all possible models in which the KB is true and checks if α is also true in these models. We list the n symbols in the KB and enumerate the 2^n models in a depth-first manner and check the truth of KB and α .

```
[121]: psource(tt_check_all)
```

<IPython.core.display.HTML object>

The algorithm basically computes every line of the truth table $KB \implies \alpha$ and checks if it is true everywhere. If symbols are defined, the routine recursively constructs every combination of truth values for the symbols and then, it checks whether `model` is consistent with `kb`. The given models correspond to the lines in the truth table, which have a `true` in the KB column, and for these lines it checks whether the query evaluates to true `result = pl_true(alpha, model)`. In short, `tt_check_all` evaluates this logical expression for each `model` `pl_true(kb, model) ==> pl_true(alpha, model)` which is logically equivalent to `pl_true(kb, model) & ~pl_true(alpha, model)` that is, the knowledge base and the negation of the query are logically inconsistent. `tt_entails()` just extracts the symbols from the query and calls `tt_check_all()` with the proper parameters.

```
[122]: psource(tt_entails)
```

<IPython.core.display.HTML object>

Keep in mind that for two symbols P and Q , $P \implies Q$ is false only when P is `True` and Q is `False`. Example usage of `tt_entails()`:

```
[123]: tt_entails(P & Q, Q)
```

[123]: True

$P \ \& \ Q$ is True only when both P and Q are True. Hence, $(P \ \& \ Q) \Rightarrow Q$ is True

```
[124]: tt_entails(P | Q, Q)
```

[124]: False

```
[125]: tt_entails(P | Q, P)
```

[125]: False

If we know that $P \mid Q$ is true, we cannot infer the truth values of P and Q . Hence $(P \mid Q) \Rightarrow Q$ is False and so is $(P \mid Q) \Rightarrow P$.

```
[126]: (A, B, C, D, E, F, G) = symbols('A, B, C, D, E, F, G')
       tt_entails(A & (B | C) & D & E & ~(F | G), A & D & E & ~F & ~G)
```

[126]: True

We can see that for the KB to be true, A, D, E have to be True and F and G have to be False. Nothing can be said about B or C .

Coming back to our problem, note that `tt_entails()` takes an `Expr` which is a conjunction of clauses as the input instead of the KB itself. You can use the `ask_if_true()` method of `PropKB` which does all the required conversions. Let's check what `wumpus_kb` tells us about $P_{1,1}$.

```
[127]: wumpus_kb.ask_if_true(~P11), wumpus_kb.ask_if_true(P11)
```

[127]: (True, False)

Looking at Figure 7.9 we see that in all models in which the knowledge base is True, $P_{1,1}$ is False. It makes sense that `ask_if_true()` returns True for $\alpha = \neg P_{1,1}$ and False for $\alpha = P_{1,1}$. This begs the question, what if α is True in only a portion of all models. Do we return True or False? This doesn't rule out the possibility of α being True but it is not entailed by the KB so we return False in such cases. We can see this is the case for $P_{2,2}$ and $P_{3,1}$.

```
[128]: wumpus_kb.ask_if_true(~P22), wumpus_kb.ask_if_true(P22)
```

[128]: (False, False)

1.8.1 Proof by Resolution

Recall that our goal is to check whether $KB \models \alpha$ i.e. is $KB \Rightarrow \alpha$ true in every model. Suppose we wanted to check if $P \Rightarrow Q$ is valid. We check the satisfiability of $\neg(P \Rightarrow Q)$, which can be rewritten as $P \wedge \neg Q$. If $P \wedge \neg Q$ is unsatisfiable, then $P \Rightarrow Q$ must be true in all models. This gives us the result “ $KB \models \alpha$ if and only if $KB \wedge \neg \alpha$ is unsatisfiable”. This technique corresponds to proof by contradiction, a standard mathematical proof technique. We assume α to be false and show that this leads to a contradiction with known axioms in KB. We obtain a contradiction by making valid inferences using inference rules. In this proof we use a single inference rule, resolution which states

$(l_1 \vee \dots \vee l_k) \wedge (m_1 \vee \dots \vee m_n) \wedge (l_i \iff \neg m_j) \implies l_1 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n$. Applying the resolution yields us a clause which we add to the KB. We keep doing this until:

- There are no new clauses that can be added, in which case $\text{KB} \not\models \alpha$.
- Two clauses resolve to yield the empty clause, in which case $\text{KB} \models \alpha$.

The empty clause is equivalent to False because it arises only from resolving two complementary unit clauses such as P and $\neg P$ which is a contradiction as both P and $\neg P$ can't be True at the same time.

There is one catch however, the algorithm that implements proof by resolution cannot handle complex sentences. Implications and bi-implications have to be simplified into simpler clauses. We already know that *every sentence of a propositional logic is logically equivalent to a conjunction of clauses*. We will use this fact to our advantage and simplify the input sentence into the **conjunctive normal form** (CNF) which is a conjunction of disjunctions of literals. For eg:

$$(A \vee B) \wedge (\neg B \vee C \vee \neg D) \wedge (D \vee \neg E)$$

This is equivalent to the POS (Product of sums) form in digital electronics. Here's an outline of how the conversion is done: 1. Convert bi-implications to implications $\alpha \iff \beta$ can be written as $(\alpha \implies \beta) \wedge (\beta \implies \alpha)$ This also applies to compound sentences $\alpha \iff (\beta \vee \gamma)$ can be written as $(\alpha \implies (\beta \vee \gamma)) \wedge ((\beta \vee \gamma) \implies \alpha)$ 2. Convert implications to their logical equivalents $\alpha \implies \beta$ can be written as $\neg \alpha \vee \beta$ 3. Move negation inwards CNF requires atomic literals. Hence, negation cannot appear on a compound statement. De Morgan's laws will be helpful here. $\neg(\alpha \wedge \beta) \equiv (\neg \alpha \vee \neg \beta)$ $\neg(\alpha \vee \beta) \equiv (\neg \alpha \wedge \neg \beta)$ 4. Distribute disjunction over conjunction Disjunction and conjunction are distributive over each other. Now that we only have conjunctions, disjunctions and negations in our expression, we will distribute disjunctions over conjunctions wherever possible as this will give us a sentence which is a conjunction of simpler clauses, which is what we wanted in the first place. We need a term of the form $(\alpha_1 \vee \alpha_2 \vee \alpha_3 \dots) \wedge (\beta_1 \vee \beta_2 \vee \beta_3 \dots) \wedge (\gamma_1 \vee \gamma_2 \vee \gamma_3 \dots) \wedge \dots$ The `to_cnf` function executes this conversion using helper subroutines.

[129]: `psource(to_cnf)`

<IPython.core.display.HTML object>

`to_cnf` calls three subroutines. `eliminate_implications` converts bi-implications and implications to their logical equivalents. `move_not_inwards` removes negations from compound statements and moves them inwards using De Morgan's laws. `distribute_and_over_or` distributes disjunctions over conjunctions. Run the cell below for implementation details.

[130]: `psource(eliminate_implications)`
`psource(move_not_inwards)`
`psource(distribute_and_over_or)`

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

Let's convert some sentences to see how it works


```
[131]: A, B, C, D = expr('A, B, C, D')
       to_cnf(A | '<=>' | B)
```

```
[131]: ((A | ~B) & (B | ~A))
```

```
[132]: to_cnf(A | '<=>' | (B & C))
```

```
[132]: ((A | ~B | ~C) & (B | ~A) & (C | ~A))
```

```
[133]: to_cnf(A & (B | (C & D)))
```

```
[133]: (A & (C | B) & (D | B))
```

```
[134]: to_cnf((A | '<=>' | ~B) | '==>' | (C | ~D))
```

```
[134]: ((B | ~A | C | ~D) & (A | ~A | C | ~D) & (B | ~B | C | ~D) & (A | ~B | C | ~D))
```

Coming back to our resolution problem, we can see how the `to_cnf` function is utilized here

```
[135]: psource(pl_resolution)
```

<IPython.core.display.HTML object>

```
[136]: pl_resolution(wumpus_kb, ~P11), pl_resolution(wumpus_kb, P11)
```

```
[136]: (True, False)
```

```
[137]: pl_resolution(wumpus_kb, ~P22), pl_resolution(wumpus_kb, P22)
```

```
[137]: (False, False)
```

1.8.2 Forward and backward chaining

Previously, we said we will look at two algorithms to check if a sentence is entailed by the KB. Here's a third one. The difference here is that our goal now is to determine if a knowledge base of definite clauses entails a single proposition symbol q - the query. There is a catch however - the knowledge base can only contain **Horn clauses**. ##### Horn Clauses Horn clauses can be defined as a *disjunction of literals* with **at most** one positive literal. A Horn clause with exactly one positive literal is called a *definite clause*. A Horn clause might look like $\neg a \vee \neg b \vee \neg c \vee \neg d \dots \vee z$ This, coincidentally, is also a definite clause. Using De Morgan's laws, the example above can be simplified to $a \wedge b \wedge c \wedge d \dots \implies z$ This seems like a logical representation of how humans process known data and facts. Assuming percepts $a, b, c, d \dots$ to be true simultaneously, we can infer z to also be true at that point in time. There are some interesting aspects of Horn clauses that make algorithmic inference or *resolution* easier. - Definite clauses can be written as implications: The most important simplification a definite clause provides is that it can be written as an implication. The premise (or the knowledge that leads to the implication) is a conjunction of positive literals. The conclusion (the implied statement) is also a positive literal. The sentence thus becomes easier to understand. The premise and the conclusion are conventionally called the *body* and the *head* respectively. A single positive literal is called a *fact*. - Forward chaining and backward chaining can be used for inference from Horn clauses: Forward chaining is semantically identical

to **AND-OR-Graph-Search** from the chapter on search algorithms. Implementational details will be explained shortly. - Deciding entailment with Horn clauses is linear in size of the knowledge base: Surprisingly, the forward and backward chaining algorithms traverse each element of the knowledge base at most once, greatly simplifying the problem. The function `pl_fc_entails` implements forward chaining to see if a knowledge base KB entails a symbol `q`. Before we proceed further, note that `pl_fc_entails` doesn't use an ordinary KB instance. The knowledge base here is an instance of the `PropDefiniteKB` class, derived from the `PropKB` class, but modified to store definite clauses. The main point of difference arises in the inclusion of a helper method to `PropDefiniteKB` that returns a list of clauses in KB that have a given symbol `p` in their premise.

```
[138]: psource(PropDefiniteKB.clauses_with_premise)
```

```
<IPython.core.display.HTML object>
```

Let's now have a look at the `pl_fc_entails` algorithm.

```
[139]: psource(pl_fc_entails)
```

```
<IPython.core.display.HTML object>
```

The function accepts a knowledge base KB (an instance of `PropDefiniteKB`) and a query `q` as inputs. `count` initially stores the number of symbols in the premise of each sentence in the knowledge base. The `conjuncts` helper function separates a given sentence at conjunctions. `inferred` is initialized as a *boolean* defaultdict. This will be used later to check if we have inferred all premises of each clause of the agenda. `agenda` initially stores a list of clauses that the knowledge base knows to be true. The `is_prop_symbol` helper function checks if the given symbol is a valid propositional logic symbol. We now iterate through `agenda`, popping a symbol `p` on each iteration. If the query `q` is the same as `p`, we know that entailment holds. The agenda is processed, reducing `count` by one for each implication with a premise `p`. A conclusion is added to the agenda when `count` reaches zero. This means we know all the premises of that particular implication to be true. `clauses_with_premise` is a helpful method of the `PropKB` class. It returns a list of clauses in the knowledge base that have `p` in their premise. Now that we have an idea of how this function works, let's see a few examples of its usage, but we first need to define our knowledge base. We assume we know the following clauses to be true.

```
[140]: clauses = ['(B & F)==>E',
                  '(A & E & F)==>G',
                  '(B & C)==>F',
                  '(A & B)==>D',
                  '(E & F)==>H',
                  '(H & I)==>J',
                  'A',
                  'B',
                  'C']
```

We will now tell this information to our knowledge base.

```
[141]: definite_clauses_KB = PropDefiniteKB()
for clause in clauses:
    definite_clauses_KB.tell(expr(clause))
```

We can now check if our knowledge base entails the following queries.

```
[142]: pl_fc_entails(definite_clauses_KB, expr('G'))
```

```
[142]: True
```

```
[143]: pl_fc_entails(definite_clauses_KB, expr('H'))
```

```
[143]: True
```

```
[144]: pl_fc_entails(definite_clauses_KB, expr('I'))
```

```
[144]: False
```

```
[145]: pl_fc_entails(definite_clauses_KB, expr('J'))
```

```
[145]: False
```

1.8.3 Effective Propositional Model Checking

The previous segments elucidate the algorithmic procedure for model checking. In this segment, we look at ways of making them computationally efficient. The problem we are trying to solve is conventionally called the *propositional satisfiability problem*, abbreviated as the *SAT* problem. In layman terms, if there exists a model that satisfies a given Boolean formula, the formula is called satisfiable. The SAT problem was the first problem to be proven *NP-complete*. The main characteristics of an NP-complete problem are: - Given a solution to such a problem, it is easy to verify if the solution solves the problem. - The time required to actually solve the problem using any known algorithm increases exponentially with respect to the size of the problem. Due to these properties, heuristic and approximal methods are often applied to find solutions to these problems. It is extremely important to be able to solve large scale SAT problems efficiently because many combinatorial problems in computer science can be conveniently reduced to checking the satisfiability of a propositional sentence under some constraints. We will introduce two new algorithms that perform propositional model checking in a computationally effective way.

1.8.4 1. DPLL (Davis-Putnam-Logeman-Loveland) algorithm

This algorithm is very similar to Backtracking-Search. It recursively enumerates possible models in a depth-first fashion with the following improvements over algorithms like `tt_entails`: 1. Early termination: In certain cases, the algorithm can detect the truth value of a statement using just a partially completed model. For example, $(P \vee Q) \wedge (P \vee R)$ is true if P is true, regardless of other variables. This reduces the search space significantly. 2. Pure symbol heuristic: A symbol that has the same sign (positive or negative) in all clauses is called a *pure symbol*. It isn't difficult to see that any satisfiable model will have the pure symbols assigned such that its parent clause becomes *true*. For example, $(P \vee \neg Q) \wedge (\neg Q \vee \neg R) \wedge (R \vee P)$ has P and Q as pure symbols and for the sentence to be true, P *has* to be true and Q *has* to be false. The pure symbol heuristic thus simplifies the problem a bit. 3. Unit clause heuristic: In the context of DPLL, clauses with just one literal and clauses with all but one *false* literals are called unit clauses. If a clause is a unit clause, it can only be satisfied by assigning the necessary value to make the last literal true. We have no other choice. Assigning one unit clause can create another unit clause. For example, when P is false, $(P \vee Q)$

becomes a unit clause, causing *true* to be assigned to Q. A series of forced assignments derived from previous unit clauses is called *unit propagation*. In this way, this heuristic simplifies the problem further. The algorithm often employs other tricks to scale up to large problems. However, these tricks are currently out of the scope of this notebook. Refer to section 7.6 of the book for more details. Let's have a look at the algorithm.

```
[146]: psource(dpll)
```

<IPython.core.display.HTML object>

The algorithm uses the ideas described above to check satisfiability of a sentence in propositional logic. It recursively calls itself, simplifying the problem at each step. It also uses helper functions `find_pure_symbol` and `find_unit_clause` to carry out steps 2 and 3 above. The `dpll_satisfiable` helper function converts the input clauses to *conjunctive normal form* and calls the `dpll` function with the correct parameters.

```
[147]: psource(dpll_satisfiable)
```

<IPython.core.display.HTML object>

Let's see a few examples of usage.

```
[148]: A, B, C, D = expr('A, B, C, D')
```

```
[149]: dpll_satisfiable(A & B & ~C & D)
```

```
[149]: {B: True, A: True, D: True, C: False}
```

This is a simple case to highlight that the algorithm actually works.

```
[150]: dpll_satisfiable((A & B) | (C & ~A) | (B & ~D))
```

```
[150]: {B: True, D: False}
```

If a particular symbol isn't present in the solution, it means that the solution is independent of the value of that symbol. In this case, the solution is independent of A.

```
[151]: dpll_satisfiable(A | '<=>' | B)
```

```
[151]: {B: True, A: True}
```

```
[152]: dpll_satisfiable((A | '<=>' | B) | '==>' | (C & ~A))
```

```
[152]: {C: True, B: True, A: False}
```

```
[153]: dpll_satisfiable((A | (B & C)) | '<=>' | ((A | B) & (A | C)))
```

```
[153]: {B: True, A: True}
```

1.8.5 2. WalkSAT algorithm

This algorithm is very similar to Hill climbing. On every iteration, the algorithm picks an unsatisfied clause and flips a symbol in the clause. This is similar to finding a neighboring state in the `hill_climbing` algorithm. The symbol to be flipped is decided by an evaluation function that counts the number of unsatisfied clauses. Sometimes, symbols are also flipped randomly to avoid local optima. A subtle balance between greediness and randomness is required. Alternatively, some versions of the algorithm restart with a completely new random assignment if no solution has been found for too long as a way of getting out of local minima of numbers of unsatisfied clauses. Let's have a look at the algorithm.

```
[154]: psource(WalkSAT)
```

<IPython.core.display.HTML object>

The function takes three arguments: 1. The `clauses` we want to satisfy. 2. The probability `p` of randomly changing a symbol. 3. The maximum number of flips (`max_flips`) the algorithm will run for. If the clauses are still unsatisfied, the algorithm returns `None` to denote failure. The algorithm is identical in concept to Hill climbing and the code isn't difficult to understand. Let's see a few examples of usage.

```
[155]: A, B, C, D = expr('A, B, C, D')
```

```
[156]: WalkSAT([A, B, ~C, D], 0.5, 100)
```

```
[156]: {B: True, D: True, C: False, A: True}
```

This is a simple case to show that the algorithm converges.

```
[157]: WalkSAT([A & B, A & C], 0.5, 100)
```

```
[157]: {B: True, C: True, A: True}
```

```
[158]: WalkSAT([A & B, C & D, C & B], 0.5, 100)
```

```
[158]: {B: True, D: True, C: True, A: True}
```

```
[159]: WalkSAT([A & B, C | D, ~(D | B)], 0.5, 1000)
```

This one doesn't give any output because WalkSAT did not find any model where these clauses hold. We can solve these clauses to see that they together form a contradiction and hence, it isn't supposed to have a solution.

One point of difference between this algorithm and the `dpll_satisfiable` algorithms is that both these algorithms take inputs differently. For WalkSAT to take complete sentences as input, we can write a helper function that converts the input sentence into conjunctive normal form and then calls WalkSAT with the list of conjuncts of the CNF form of the sentence.

```
[160]: def WalkSAT_CNF(sentence, p=0.5, max_flips=10000):  
        return WalkSAT(conjuncts(to_cnf(sentence)), 0, max_flips)
```

Now we can call `WalkSAT_CNF` and `DPLL_Satisfiable` with the same arguments.

```
[161]: WalkSAT_CNF((A & B) | (C & ~A) | (B & ~D), 0.5, 1000)
```

```
[161]: {B: False, D: True, C: True, A: False}
```

It works! Notice that the solution generated by WalkSAT doesn't omit variables that the sentence doesn't depend upon. If the sentence is independent of a particular variable, the solution contains a random value for that variable because of the stochastic nature of the algorithm. Let's compare the runtime of WalkSAT and DPLL for a few cases. We will use the %%timeit magic to do this.

```
[162]: sentence_1 = A | '<=>' | B
       sentence_2 = (A & B) | (C & ~A) | (B & ~D)
       sentence_3 = (A | (B & C)) | '<=>' | ((A | B) & (A | C))
```

```
[163]: %%timeit
       dpll_satisfiable(sentence_1)
       dpll_satisfiable(sentence_2)
       dpll_satisfiable(sentence_3)
```

183 s ± 4.66 s per loop (mean ± std. dev. of 7 runs, 1,000 loops each)

```
[164]: %%timeit
       WalkSAT_CNF(sentence_1)
       WalkSAT_CNF(sentence_2)
       WalkSAT_CNF(sentence_3)
```

134 s ± 634 ns per loop (mean ± std. dev. of 7 runs, 10,000 loops each)

On an average, for solvable cases, WalkSAT is quite faster than dpll because, for a small number of variables, WalkSAT can reduce the search space significantly. Results can be different for sentences with more symbols though. Feel free to play around with this to understand the trade-offs of these algorithms better.

1.8.6 SATPlan

In this section we show how to make plans by logical inference. The basic idea is very simple. It includes the following three steps: 1. Construct a sentence that includes: 1. A collection of assertions about the initial state. 2. The successor-state axioms for all the possible actions at each time up to some maximum time t. 3. The assertion that the goal is achieved at time t. 2. Present the whole sentence to a SAT solver. 3. Assuming a model is found, extract from the model those variables that represent actions and are assigned true. Together they represent a plan to achieve the goals.

Lets have a look at the algorithm

```
[165]: psource(SAT_plan)
```

<IPython.core.display.HTML object>

Let's see few examples of its usage. First we define a transition and then call SAT_plan.

```
[166]: transition = {'A': {'Left': 'A', 'Right': 'B'},
                  'B': {'Left': 'A', 'Right': 'C'},
```

```
'C': {'Left': 'B', 'Right': 'C'}}
```

```
print(SAT_plan('A', transition, 'C', 2))
print(SAT_plan('A', transition, 'B', 3))
print(SAT_plan('C', transition, 'A', 3))
```

```
['Right', 'Right']
['Right']
['Left', 'Left']
```

Let us do the same for another transition.

```
[167]: transition = {(0, 0): {'Right': (0, 1), 'Down': (1, 0)},
                    (0, 1): {'Left': (1, 0), 'Down': (1, 1)},
                    (1, 0): {'Right': (1, 0), 'Up': (1, 0), 'Left': (1, 0), 'Down': (1, 0)},
                    (1, 1): {'Left': (1, 0), 'Up': (0, 1)}}

print(SAT_plan((0, 0), transition, (1, 1), 4))
```

```
['Right', 'Down']
```

1.9 First-Order Logic Knowledge Bases: FolKB

The class `FolKB` can be used to represent a knowledge base of First-order logic sentences. You would initialize and use it the same way as you would for `PropKB` except that the clauses are first-order definite clauses. We will see how to write such clauses to create a database and query them in the following sections.

1.10 Criminal KB

In this section we create a `FolKB` based on the following paragraph. The law says that it is a crime for an American to sell weapons to hostile nations. The country Nono, an enemy of America, has some missiles, and all of its missiles were sold to it by Colonel West, who is American. The first step is to extract the facts and convert them into first-order definite clauses. Extracting the facts from data alone is a challenging task. Fortunately, we have a small paragraph and can do extraction and conversion manually. We'll store the clauses in list aptly named `clauses`.

```
[168]: clauses = []
```

“... it is a crime for an American to sell weapons to hostile nations” The keywords to look for here are ‘crime’, ‘American’, ‘sell’, ‘weapon’ and ‘hostile’. We use predicate symbols to make meaning of them.

- `Criminal(x)`: x is a criminal
- `American(x)`: x is an American
- `Sells(x, y, z)`: x sells y to z
- `Weapon(x)`: x is a weapon

- **Hostile(x)**: x is a hostile nation

Let us now combine them with appropriate variable naming to depict the meaning of the sentence. The criminal x is also the American x who sells weapon y to z, which is a hostile nation.

$American(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z) \implies Criminal(x)$

```
[169]: clauses.append(expr("(American(x) & Weapon(y) & Sells(x, y, z) & Hostile(z))  
       $\hookrightarrow$ ==> Criminal(x)"))
```

“The country Nono, an enemy of America” We now know that Nono is an enemy of America. We represent these nations using the constant symbols **Nono** and **America**. the enemy relation is show using the predicate symbol **Enemy**.

Enemy(Nono, America)

```
[170]: clauses.append(expr("Enemy(Nono, America)"))
```

“Nono ... has some missiles” This states the existence of some missile which is owned by Nono. $\exists x Owns(Nono, x) \wedge Missile(x)$. We invoke existential instantiation to introduce a new constant **M1** which is the missile owned by Nono.

Owns(Nono, M1), Missile(M1)

```
[171]: clauses.append(expr("Owns(Nono, M1)"))  
      clauses.append(expr("Missile(M1)"))
```

“All of its missiles were sold to it by Colonel West” If Nono owns something and it classifies as a missile, then it was sold to Nono by West.

$Missile(x) \wedge Owns(Nono, x) \implies Sells(West, x, Nono)$

```
[172]: clauses.append(expr("(Missile(x) & Owns(Nono, x)) ==> Sells(West, x, Nono)"))
```

“West, who is American” West is an American.

American(West)

```
[173]: clauses.append(expr("American(West)"))
```

We also know, from our understanding of language, that missiles are weapons and that an enemy of America counts as “hostile”.

$Missile(x) \implies Weapon(x), Enemy(x, America) \implies Hostile(x)$

```
[174]: clauses.append(expr("Missile(x) ==> Weapon(x)"))  
      clauses.append(expr("Enemy(x, America) ==> Hostile(x)"))
```

Now that we have converted the information into first-order definite clauses we can create our first-order logic knowledge base.

```
[175]: crime_kb = FolKB(classes)
```

The **subst** helper function substitutes variables with given values in first-order logic statements. This will be useful in later algorithms. It’s implementation is quite simple and self-explanatory.


```
[176]: psource(subst)
```

```
<IPython.core.display.HTML object>
```

Here's an example of how `subst` can be used.

```
[177]: subst({x: expr('Nono'), y: expr('M1')}, expr('Owns(x, y)'))
```

```
[177]: Owns(Nono, M1)
```

1.11 Inference in First-Order Logic

In this section we look at a forward chaining and a backward chaining algorithm for `FolKB`. Both aforementioned algorithms rely on a process called unification, a key component of all first-order inference algorithms.

1.11.1 Unification

We sometimes require finding substitutions that make different logical expressions look identical. This process, called unification, is done by the `unify` algorithm. It takes as input two sentences and returns a unifier for them if one exists. A unifier is a dictionary which stores the substitutions required to make the two sentences identical. It does so by recursively unifying the components of a sentence, where the unification of a variable symbol `var` with a constant symbol `Const` is the mapping `{var: Const}`. Let's look at a few examples.

```
[178]: unify(expr('x'), 3)
```

```
[178]: {x: 3}
```

```
[179]: unify(expr('A(x)'), expr('A(B)'))
```

```
[179]: {x: B}
```

```
[180]: unify(expr('Cat(x) & Dog(Dobby)'), expr('Cat(Bella) & Dog(y)'))
```

```
[180]: {x: Bella, y: Dobby}
```

In cases where there is no possible substitution that unifies the two sentences the function return `None`.

```
[181]: print(unify(expr('Cat(x)'), expr('Dog(Dobby)')))
```

```
None
```

We also need to take care we do not unintentionally use the same variable name. `Unify` treats them as a single variable which prevents it from taking multiple value.

```
[182]: print(unify(expr('Cat(x) & Dog(Dobby)'), expr('Cat(Bella) & Dog(x)')))
```

```
None
```

1.11.2 Forward Chaining Algorithm

We consider the simple forward-chaining algorithm presented in Figure 9.3. We look at each rule in the knowledge base and see if the premises can be satisfied. This is done by finding a substitution which unifies each of the premise with a clause in the KB. If we are able to unify the premises, the conclusion (with the corresponding substitution) is added to the KB. This inferencing process is repeated until either the query can be answered or till no new sentences can be added. We test if the newly added clause unifies with the query in which case the substitution yielded by `unify` is an answer to the query. If we run out of sentences to infer, this means the query was a failure.

The function `fol_fc_ask` is a generator which yields all substitutions which validate the query.

```
[183]: psource(fol_fc_ask)
```

<IPython.core.display.HTML object>

Let's find out all the hostile nations. Note that we only told the KB that Nono was an enemy of America, not that it was hostile.

```
[184]: answer = fol_fc_ask(crime_kb, expr('Hostile(x)'))
       print(list(answer))
```

```
[{x: Nono}]
```

The generator returned a single substitution which says that Nono is a hostile nation. See how after adding another enemy nation the generator returns two substitutions.

```
[185]: crime_kb.tell(expr('Enemy(JaJa, America)'))
       answer = fol_fc_ask(crime_kb, expr('Hostile(x)'))
       print(list(answer))
```

```
[{x: Nono}, {x: JaJa}]
```

Note: `fol_fc_ask` makes changes to the KB by adding sentences to it.

1.11.3 Backward Chaining Algorithm

This algorithm works backward from the goal, chaining through rules to find known facts that support the proof. Suppose `goal` is the query we want to find the substitution for. We find rules of the form `lhs  $\implies$  goal` in the KB and try to prove `lhs`. There may be multiple clauses in the KB which give multiple `lhs`. It is sufficient to prove only one of these. But to prove a `lhs` all the conjuncts in the `lhs` of the clause must be proved. This makes it similar to And/Or search.

OR The OR part of the algorithm comes from our choice to select any clause of the form `lhs  $\implies$  goal`. Looking at all rules's `lhs` whose `rhs` unify with the `goal`, we yield a substitution which proves all the conjuncts in the `lhs`. We use `parse_definite_clause` to attain `lhs` and `rhs` from a clause of the form `lhs  $\implies$  rhs`. For atomic facts the `lhs` is an empty list.

```
[186]: psource(fol_bc_or)
```

<IPython.core.display.HTML object>

AND The AND corresponds to proving all the conjuncts in the `lhs`. We need to find a substitution which proves each and every clause in the list of conjuncts.

```
[187]: psource(fol_bc_and)
```

<IPython.core.display.HTML object>

Now the main function `fl_bc_ask` calls `fol_bc_or` with substitution initialized as empty. The `ask` method of `FolKB` uses `fol_bc_ask` and fetches the first substitution returned by the generator to answer query. Let's query the knowledge base we created from `clauses` to find hostile nations.

```
[188]: # Rebuild KB because running fol_fc_ask would add new facts to the KB
crime_kb = FolKB(clauses)
```

```
[189]: crime_kb.ask(expr('Hostile(x)'))
```

```
[189]: {v_14: Nono, x: Nono}
```

You may notice some new variables in the substitution. They are introduced to standardize the variable names to prevent naming problems as discussed in the [Unification section](#)

1.12 Appendix: The Implementation of `'==>'`

Consider the `Expr` formed by this syntax:

```
[190]: P | '==>' | ~Q
```

```
[190]: (P ==> ~Q)
```

What is the funny `'==>'` syntax? The trick is that `"|"` is just the regular Python or-operator, and so is exactly equivalent to this:

```
[191]: (P | '==>') | ~Q
```

```
[191]: (P ==> ~Q)
```

In other words, there are two applications of or-operators. Here's the first one:

```
[192]: P | '==>'
```

```
[192]: PartialExpr('==>', P)
```

What is going on here is that the `__or__` method of `Expr` serves a dual purpose. If the right-hand-side is another `Expr` (or a number), then the result is an `Expr`, as in `(P | Q)`. But if the right-hand-side is a string, then the string is taken to be an operator, and we create a node in the abstract syntax tree corresponding to a partially-filled `Expr`, one where we know the left-hand-side is `P` and the operator is `==>`, but we don't yet know the right-hand-side.

The `PartialExpr` class has an `__or__` method that says to create an `Expr` node with the right-hand-side filled in. Here we can see the combination of the `PartialExpr` with `Q` to create a complete `Expr`:

```
[193]: partial = PartialExpr('==>', P)
partial | ~Q
```

```
[193]: (P ==> ~Q)
```

This [trick](#) is due to [Ferdinand Jamitzky](#), with a modification by [C. G. Vedant](#), who suggested using a string inside the or-bars.

1.13 Appendix: The Implementation of `expr`

How does `expr` parse a string into an `Expr`? It turns out there are two tricks (besides the Jamitzky/Vedant trick):

1. We do a string substitution, replacing “`==>`” with “`| '==>' |`” (and likewise for other operators).
2. We `eval` the resulting string in an environment in which every identifier is bound to a symbol with that identifier as the `op`.

In other words,

```
[194]: expr('~(P & Q) ==> (~P | ~Q)')
```

```
[194]: (~(P & Q) ==> (~P | ~Q))
```

is equivalent to doing:

```
[195]: P, Q = symbols('P, Q')
~(P & Q) | '==>' | (~P | ~Q)
```

```
[195]: (~(P & Q) ==> (~P | ~Q))
```

One thing to beware of: this puts `==>` at the same precedence level as `|`, which is not quite right. For example, we get this:

```
[196]: P & Q | '==>' | P | Q
```

```
[196]: (((P & Q) ==> P) | Q)
```

which is probably not what we meant; when in doubt, put in extra parens:

```
[197]: (P & Q) | '==>' | (P | Q)
```

```
[197]: ((P & Q) ==> (P | Q))
```

```
[3]: from logic import *

# we'll focus only on the relevant cells
# define our knowledge base parts
kb_components = []

# breeze-pit relationships (only for relevant cells)
```

```

kb_components.append(equiv(breeze(1, 1), pit(1, 2) | pit(2, 1)))
kb_components.append(equiv(breeze(2, 1), pit(1, 1) | pit(2, 2) | pit(3, 1)))
kb_components.append(equiv(breeze(1, 2), pit(1, 1) | pit(1, 3) | pit(2, 2)))

# stench-wumpus relationships (only for relevant cells)
kb_components.append(equiv(stench(1, 1), wumpus(1, 2) | wumpus(2, 1)))
kb_components.append(equiv(stench(2, 1), wumpus(1, 1) | wumpus(2, 2) |
    ↪wumpus(3, 1)))
kb_components.append(equiv(stench(1, 2), wumpus(1, 1) | wumpus(1, 3) |
    ↪wumpus(2, 2)))

# only one wumpus exists (simplified to relevant cells)
kb_components.append((wumpus(1, 1) | wumpus(1, 2) | wumpus(1, 3) |
    wumpus(2, 1) | wumpus(2, 2) | wumpus(3, 1)))

# observed percepts
kb_components.append(~breeze(1, 1)) # no breeze in [1,1]
kb_components.append(~stench(1, 1)) # no stench in [1,1]
kb_components.append(breeze(2, 1)) # breeze in [2,1]
kb_components.append(stench(1, 2)) # stench in [1,2]

# combine into a single formula
kb = kb_components[0]
for clause in kb_components[1:]:
    kb = kb & clause

# function to check if alpha is entailed using model checking
def check_entailment(kb, alpha):
    # kb != alpha is equivalent to kb & ~alpha being unsatisfiable
    negated_alpha = ~alpha
    formula_to_check = kb & negated_alpha

    # if kb & ~alpha is unsatisfiable, then kb != alpha
    return not dpll_satisfiable(formula_to_check)

# propositions we want to check
alpha2 = ~pit(2, 2) # "there is no pit in [2,2]"
alpha3 = wumpus(1, 3) # "there is a wumpus in [1,3]"

# check entailment
print(f"kb != 'there is no pit in [2,2]': {check_entailment(kb, alpha2)}")
print(f"kb != 'there is a pit in [2,2]': {check_entailment(kb, ~alpha2)}")
print(f"kb != 'there is a wumpus in [1,3]': {check_entailment(kb, alpha3)}")
print(f"KB != 'There is no Wumpus in [1,3]': {check_entailment(kb, ~alpha3)}")

```

```
kb != 'there is no pit in [2,2]': False
```

```
kb |= 'there is a pit in [2,2]': False
kb |= 'there is a wumpus in [1,3]': False
KB |= 'There is no Wumpus in [1,3]': False
```

2 Authors

This notebook by [Chirag Vartak](#) and [Peter Norvig](#).